I checked your uploaded Python code, ATM Transaction.py. I'll prepare a complete activity record based on your actual program, using simple language suitable for a B.Tech 1st-year CSE Data Science student.

Your program manages ATM transactions by accepting an opening balance, processing deposits and withdrawals, checking transaction amounts and available balance, and displaying the transaction history and final balance.

ATM Transaction Management System

1. Problem statement

To develop a Python program that simulates basic ATM transactions. The program should allow users to deposit money and withdraw money from their account. It must validate transaction types, ensure that the amount is positive, prevent withdrawals exceeding the available balance, and display the transaction history and final account balance.

2. Objectives

1. To understand the use of variables and data types in Python.
2. To perform deposit and withdrawal operations.
3. To use conditional statements for transaction validation.
4. To use loops to process multiple transactions.
5. To store transaction details using a list.
6. To display transaction history and the final balance.

3. Assumptions

- The user enters a valid numerical opening balance.
- The transaction type must be D for deposit, W for withdrawal, or 0 to finish.
- The transaction amount must be greater than zero.
- A withdrawal is allowed only when sufficient balance is available.
- The program runs in the Python console.
- The program maintains transaction details only while it is running.

4. Algorithm

1. Start the program.
2. Read the opening balance from the user.
3. Create an empty list to store transactions.
4. Ask the user to enter a transaction type: deposit, withdrawal, or finish.
5. If the user enters 0, stop accepting transactions.
6. If the transaction type is invalid, display an error message and ask again.
7. Read the transaction amount.
8. If the amount is zero or negative, display an error message.

9. If the transaction is a deposit, add the amount to the balance and record the transaction.
10. If the transaction is a withdrawal, check whether the balance is sufficient.
11. If sufficient, subtract the amount and record the transaction. Otherwise, reject the withdrawal.
12. Display all successful transactions and the final balance.
13. Stop the program.

5. Flowchart

Start

Enter opening balance

Enter transaction type

Is type 0?

Yes

Display transactions and final balance

End

No

Validate transaction type

Enter amount and validate

Deposit or check withdrawal balance

Update balance if successful; repeat

Note: This is a simplified flowchart. Invalid transaction types and invalid amounts return to the transaction loop.

6. Pseudocode

```
BEGIN
  INPUT opening_balance
  balance = opening_balance
  transactions = empty list

  REPEAT
    INPUT transaction_type

    IF transaction_type = "0"
      EXIT LOOP
```

```
        END IF

        IF transaction_type is not "D" or "W"
            DISPLAY "Invalid transaction type"
            CONTINUE
        END IF

        INPUT amount

        IF amount <= 0
            DISPLAY "Amount must be positive"

        ELSE IF transaction_type = "D"
            balance = balance + amount
            ADD deposit to transactions
            DISPLAY "Deposit successful"

        ELSE IF amount <= balance
            balance = balance - amount
            ADD withdrawal to transactions
            DISPLAY "Withdrawal successful"

        ELSE
            DISPLAY "Insufficient balance"
        END IF

    UNTIL finished

    DISPLAY transaction history
    DISPLAY final balance
END
```

7. Python source code

```
balance = float(input("Opening balance: "))
transactions = []

while True:
    kind = input(
        "Enter D=deposit, W=withdraw, 0=finish: "
    ).upper()

    if kind == "0":
        break

    if kind != "D" and kind != "W":
        print("Invalid transaction type.")
        continue

    amount = float(input("Enter amount: "))

    if amount <= 0:
        print("Amount must be positive.")

    elif kind == "D":
        balance += amount
        transactions.append(("Deposit", amount))
        print("Deposit successful.")

    elif amount <= balance:
        balance -= amount
        transactions.append(("Withdrawal", amount))
        print("Withdrawal successful.")

    else:
```

```
        print("Rejected: insufficient balance.")


print("\\nTransactions:")


for kind, amount in transactions:
    print(kind, "₹", round(amount, 2))


print("Final balance: ₹", round(balance, 2))
```

8. Test cases

Assume the opening balance is ₹5,000 for the following tests.

Test	Input / operation	Expected result
1	Deposit ₹1,000	Balance becomes ₹6,000
2	Withdraw ₹500	Balance becomes ₹4,500
3	Withdraw ₹6,000	Withdrawal rejected
4	Deposit ₹0	Amount must be positive
5	Enter transaction type X	Invalid transaction type
6	Enter 0	Stop transactions and display summary

Each test should be run separately, starting with an opening balance of ₹5,000.

9. Sample output screenshots

Here are sample console outputs you can use to verify your program. They are text previews, not captured screenshots from a running Python console.

Test 1 — Successful deposit and withdrawal

```
Opening balance: 5000
Enter D=deposit, W=withdraw, 0=finish: D
Enter amount: 1000
Deposit successful.
Enter D=deposit, W=withdraw, 0=finish: W
Enter amount: 500
Withdrawal successful.
Enter D=deposit, W=withdraw, 0=finish: 0

Transactions:
Deposit ₹ 1000.0
Withdrawal ₹ 500.0
Final balance: ₹ 5500.0
```

Copy sample output

Test 2 — Insufficient balance

Opening balance: 5000
Enter D=deposit, W=withdraw, 0=finish: W
Enter amount: 6000
Rejected: insufficient balance.
Enter D=deposit, W=withdraw, 0=finish: 0

Transactions:
Final balance: ₹ 5000.0

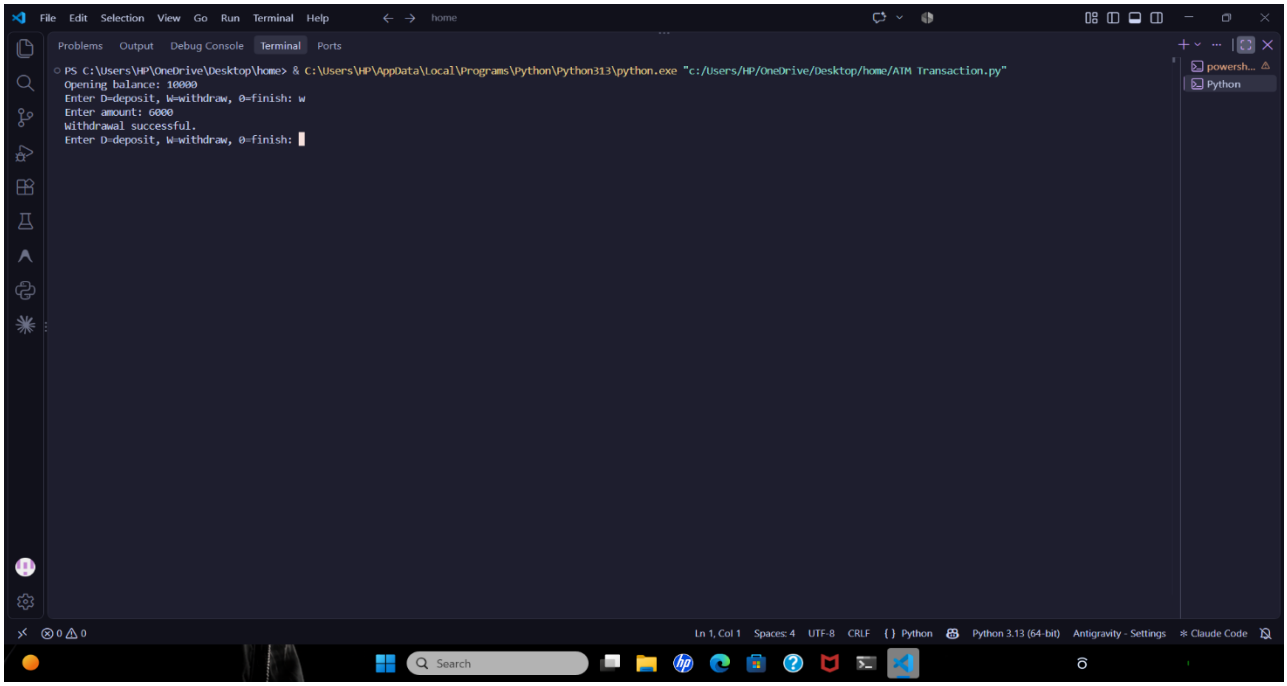
Copy sample output

To obtain actual screenshots for your record, run each test in IDLE, VS Code, or your Python terminal and take a screenshot of the output.

Small correction for neat output: Your original code uses print("\\nTransactions:"), which displays the characters \n rather than starting on a new line. Replace it with:

print("\nTransactions:")

This is only a formatting correction; it does not change the transaction logic.



10. Conclusion

The ATM Transaction Management System was developed successfully using Python. It performs deposits and withdrawals, validates transaction inputs, checks the available balance, and records successful transactions. It also displays the transaction history and final balance. This program helps in understanding loops, conditional statements, lists, variables, and basic input/output operations in Python.